

Project 1 - Lexer

Deadline: 10 月 7 日 23:59

不接受任何理由的迟交，迟交原则上不予受理。

成绩以最后一次提交为准。

每个小组仅需一位同学提交。

1 项目目标

SPL 是一门用于本课程教学的类 C 语言。在本项目中，你需要使用 ANTLR4 为 SPL 实现词法分析器 (Lexer)。

词法分析器负责读取 SPL 源程序中的字符流，并将其划分为一系列词法单元 (Token)。你需要在 `SplLexer.g4` 文件中，根据本项目给出的词法规范完成相应的 Lexer Rule，使 ANTLR 能够正确识别 SPL 中的关键字、运算符、分隔符、标识符、整数、空白符和注释。

本 Project 的重点是：

- 根据给定的词法规范正确书写 Lexer Rule；
- 理解 ANTLR Lexer 的最长匹配 (Longest Match) 规则；
- 理解 Lexer Rule 的优先级；
- 使用 `skip` 正确处理空白符和注释。

本项目**仅关注词法分析**。

你不需要判断生成的 Token 序列是否能够组成一个语法正确的 SPL 程序，也不需要自行实现额外的词法错误检测或错误恢复机制。

程序的语法结构、表达式优先级、结合性以及其它语义约束将在后续 Project 中处理。

2 SPL 词法规范

本节给出本 Project 中需要识别的全部 Token。

Token 的名称是评分的一部分，因此请严格使用下文规定的名称。

2.1 Keywords

SPL 定义以下关键字：

INT	匹配关键字 <code>int</code> 。
STRUCT	匹配关键字 <code>struct</code> 。
RETURN	匹配关键字 <code>return</code> 。
IF	匹配关键字 <code>if</code> 。
ELSE	匹配关键字 <code>else</code> 。
WHILE	匹配关键字 <code>while</code> 。

需要注意，关键字本身也满足一般标识符的字符组成规则。

例如：

```
int
integer
int_value
```

应分别识别为：

```
INT
Identifier
Identifier
```

其中，`integer` 应被识别为一个完整的 `Identifier`，而不能被拆分为一个 `INT` 和其他 `Token`。

2.2 Operators

SPL 定义以下运算符：

ASSIGN	匹配赋值运算符 <code>=</code> 。
PLUS	匹配加法运算符 <code>+</code> 。
MINUS	匹配减法运算符 <code>-</code> 。
MUL	匹配乘法运算符 <code>*</code> 。
DIV	匹配除法运算符 <code>/</code> 。
MOD	匹配取模运算符 <code>%</code> 。
POW	匹配幂运算符 <code>^</code> 。
LT	匹配小于运算符 <code><</code> 。
LE	匹配小于等于运算符 <code><=</code> 。
GT	匹配大于运算符 <code>></code> 。
GE	匹配大于等于运算符 <code>>=</code> 。
EQ	匹配等于运算符 <code>==</code> 。
NEQ	匹配不等于运算符 <code>!=</code> 。
AND	匹配逻辑与运算符 <code>&&</code> 。
OR	匹配逻辑或运算符 <code> </code> 。
NOT	匹配逻辑非运算符 <code>!</code> 。
DOT	匹配成员访问符 <code>.</code> 。

部分运算符具有相同的前缀，例如：

```
=    ==
<    <=
>    >=
!    !=
```

你需要保证这些运算符均能够被正确识别。

2.3 Separators

SPL 定义以下分隔符：

SEMI	匹配分号 ;。
COMMA	匹配逗号 ,。
LPAREN	匹配左圆括号 (。
RPAREN	匹配右圆括号)。
LBRACE	匹配左花括号 {。
RBRACE	匹配右花括号 }。
LBRACK	匹配左方括号 [。
RBRACK	匹配右方括号]。

2.4 Identifier

Identifier 用于匹配 SPL 中的标识符。

一个合法的 **Identifier** 满足：

- 必须以字母或下划线开头；
- 后续可以包含任意数量的字母、数字或下划线；
- 不能为空串。

例如：

```
a
value
_value
value1
int_value
matrix2
```

均为合法的 **Identifier**。

需要注意，SPL 中已经定义的关键字应被识别为对应的 **Keyword Token**，而不是 **Identifier**。

2.5 Number

Number 用于匹配十进制非负整数。

一个合法的 **Number** 满足：

- 数字 0 本身是合法的 **Number**；
- 当 **Number** 包含两位或更多位数字时，其第一位不能为 0；
- 本 Project 不支持浮点数；
- 本 Project 不支持十六进制、八进制或二进制整数；
- 本 Project 不支持科学计数法。

例如：

```
0
1
10
123
10086
```

均为合法的 **Number**。

2.6 Whitespaces

WS 用于匹配 SPL 中的空白字符，例如：

- 空格 (' ');
- 制表符 ('\t');
- 换行符 ('\r'、'\n')。

所有空白字符均应被 Lexer 跳过，不应出现在最终的 Token Stream 中。

2.7 Comments

SPL 支持单行注释和块注释。

2.7.1 Line Comment

LINE_COMMENT 匹配以：

```
//
```

开始的单行注释。

单行注释持续到当前行结束。如当前行为文件最后一行，则持续到文件结束 (EOF)。

例如：

```
int value = 10; // This is a comment
```

其中：

```
// This is a comment
```

应被 Lexer 跳过。

2.7.2 Block Comment

BLOCK_COMMENT 匹配以：

```
/*
```

开始，并以其后**最近出现的**：

```
*/
```

结束的块注释。

例如：

```
/*  
 * This is a block comment.  
 */  
int value = 10;
```

块注释可以跨越多行。

本 Project 中的块注释不支持嵌套。也就是说，从 /* 开始后，遇到其后第一个 */ 时，该块注释立即结束；注释内部出现的另一个 /* 不会开启新的嵌套层级。

所有块注释均应被 Lexer 跳过，不应进入最终的 Token Stream。

3 关于 Token 的划分

在本 Project 中，你只需要根据前文给出的词法规范定义各类 Token。

你不需要额外判断多个相邻 Token 组合在一起是否看起来像某种非法的 Number 或 Identifier，也不需要为此自行添加额外的错误处理规则。

例如，根据本 Project 对 Number 的定义：

```
01234
```

不能被识别为一个完整的 Number。

但是 ANTLR 可以将其划分为两个合法 Token：

```
Token: Number, Raw: 0
Token: Number, Raw: 1234
```

类似地：

```
123_awe
```

可以被划分为：

```
Token: Number, Raw: 123
Token: Identifier, Raw: _awe
```

例如：

```
123abc
```

也可以被划分为：

```
Token: Number, Raw: 123
Token: Identifier, Raw: abc
```

这些情况不需要由 Lexer 额外报告错误。

Lexer 的职责是根据词法规则将输入字符流划分成 Token。生成的 Token 序列是否能够组成合法的 SPL 程序，将由后续的 Parser 判断。

例如：

```
1 + + + 2
```

对于 Lexer 而言，可以正常产生：

```
Number
PLUS
PLUS
PLUS
Number
```

至于这一 Token 序列是否符合 SPL 的语法规则，不属于本 Project 的任务范围。

4 关于 Lexer Error

如果输入中的某个字符无法被任何 Lexer Rule 匹配，ANTLR Lexer 本身会按照其默认机制报告词法错误，并尝试继续处理后续输入。

本 Project 不要求你自行实现额外的 Lexer Error Handling，也不要求你自定义错误信息的格式。

因此，在完成本 Project 时，你只需要关注前文给出的 Lexer Rule：

- 不需要自行实现 Error Listener;
- 不需要重写 ANTLR 的错误恢复逻辑;
- 不需要为非法 Number 或 Identifier 设计额外错误类型;
- 不需要添加专门用于捕获所有剩余字符的错误规则。

除完成本项目规定的 Lexer Rule 外, 请不要额外改变 ANTLR 默认的 Lexer Error Handling 行为。

5 初始代码

Project 1 的初始代码位于课程 GitHub 仓库:

<https://github.com/sqlab-sustech/CS323-Compilers-2026F-Projects.git>

本 Project 使用 project1 分支作为 Starter Code。

5.1 获取初始代码

如果你还没有下载课程 Projects 仓库, 可以运行:

```
git clone https://github.com/sqlab-sustech/CS323-Compilers-2026F-Projects.git
cd CS323-Compilers-2026F-Projects
git checkout project1
```

如果你已经在之前的 Lab 或 Project 中下载过该仓库, 可以进入已有仓库, 并运行:

```
git fetch
git checkout project1
```

进入 project1 分支后, 你将获得完成本 Project 所需的 Starter Code, 包括:

- SplLexer.g4;
- Main.java;
- Grader.java;
- Makefile;
- ANTLR 4.13.2;

5.2 需要修改的文件

本 Project 中, 你主要需要修改:

SplLexer.g4

Starter Code 中已经提供 grammar 声明:

```
lexer grammar SplLexer;
```

请保持文件名和 grammar 名称不变, 并在此基础上完成本 Project 要求的 Lexer Rule。
ANTLR 自动生成的 Java 文件不需要也不应该手工修改。

6 Framework 说明

在 Starter Code 中，我们提供了 Project 1 的 Grader。其核心逻辑如下：

```
CharStream input = CharStreams.fromStream(sourceStream);

SplLexer lexer = new SplLexer(input);

CommonTokenStream tokens = new CommonTokenStream(lexer);

tokens.fill();

Vocabulary vocabulary = lexer.getVocabulary();

for (Token token : tokens.getTokens()) {
    this.writer.append(String.format(
        "Token: %s, Raw: %s\n",
        vocabulary.getSymbolicName(token.getType()),
        token.getText()
    ));
}
```

ANTLR 会根据你编写的 `SplLexer.g4` 自动生成对应的 Lexer 类 `SplLexer`。

Framework 首先将 SPL 源程序转换为字符流 (`CharStream`)，随后使用 `SplLexer` 进行词法分析，并生成 `Token Stream`。

之后，Grader 会依次输出每一个 Token 的：

1. Token 类型；
2. Token 在源程序中对应的原始文本 (Lexeme)。

例如：

```
Token: INT, Raw: int
Token: Identifier, Raw: main
Token: LPAREN, Raw: (
```

对于通过 `skip` 跳过的 Lexer Rule，例如：

```
WS
LINE_COMMENT
BLOCK_COMMENT
```

其对应内容不会进入最终的 `Token Stream`，因此也不会出现在输出中。

对于输入结束位置，ANTLR 会产生 EOF Token：

```
Token: EOF, Raw: <EOF>
```

你需要保证正常 Token 的类型、Lexeme 和顺序与参考结果完全一致。

7 编译和运行

Project 1 已经提供完整的 `Makefile`。

在正常情况下，你不需要自己执行 ANTLR、`javac` 或 `jar` 的底层命令。

下面的所有命令均应在 Project 根目录中执行。

7.1 构建 Project

运行：

```
make
```

即可构建整个 Project。

也可以显式运行：

```
make build
```

两者效果相同。

构建过程会自动完成：

1. 根据 `SplLexer.g4` 使用 ANTLR 生成 Java Lexer；
2. 编译生成的 Lexer 和课程提供的 Java Framework；
3. 将编译结果打包为 `target/project1.jar`。

因此，每次修改 `SplLexer.g4` 后，都可以重新运行 `make` 检查 Lexer 是否能够正常生成和编译。如果你的 Lexer Rule 存在 ANTLR grammar 语法错误，构建过程会直接显示相应的错误信息。

7.2 运行 Project

运行：

```
make run
```

即可构建并运行课程提供的 Main。

`make run` 已经依赖于 `build`，因此你不需要提前单独执行 `make build`。

你可以修改 `Main.java` 中用于本地测试的内容，构造自己的 SPL 输入来测试 Lexer。

例如：

```
Grader grader = new Grader("1 + 2");  
grader.run();
```

对 `Main.java` 的修改仅用于本地调试。

最终 OJ 评分只使用你提交的 `SplLexer.g4`。

7.3 运行公开测试

课程会在 `testcases/project1/in` 目录中提供公开测试输入，并在 `testcases/project1/out` 目录中提供对应的标准输出。

运行：

```
make test
```

即可构建 Project 并运行全部公开测试。

对于每一个测试样例，如果实际输出与标准输出完全一致，你会看到类似：

```
[PASS] case1
```

的结果。

如果测试失败，则会显示：

```
[FAIL] case1
```

并给出实际输出和标准输出之间的差异，帮助你进行本地调试。

所有测试结束后，会显示类似：

```
Result: 5 / 5 passed
```

的汇总结果。

需要注意，公开测试只覆盖本 Project 的部分情况。即使所有公开测试均通过，也不能保证隐藏测试全部通过。你仍然应该根据本 Project 的完整词法规范自行设计边界测试。

7.4 清理生成文件

运行：

```
make clean
```

会删除：

```
target/  
src/main/java/generated/
```

目录中的 ANTLR 生成文件和编译产物。

如果你怀疑之前生成的 Lexer 或编译结果影响了当前测试，可以执行：

```
make clean  
make
```

重新生成并构建整个 Project。

7.5 其他 Make Target

Makefile 还提供一些更细粒度的构建 Target。

运行：

```
make generate
```

只执行 ANTLR 代码生成。

运行：

```
make compile
```

会重新生成 Lexer，并编译所有 Java 代码。

通常情况下，你不需要单独使用这些命令。完成本 Project 时主要使用：

```
make  
make run  
make test  
make clean
```

即可。

7.6 推荐的开发流程

推荐按照以下流程完成 Project 1:

1. 获取课程仓库并切换到 `project1` 分支;
2. 阅读本 Project 的 SPL 词法规范;
3. 修改 `SplLexer.g4`;
4. 运行 `make` 检查是否能够正常生成和编译;
5. 使用 `make run` 测试自己构造的输入;
6. 使用 `make test` 运行课程提供的公开测试;
7. 根据词法规范自行补充边界测试;
8. 确认本地结果正确后, 将 `SplLexer.g4` 上传至 Hydro OJ。

8 Hydro Online Judge 使用说明

本 Project 使用课程提供的 Hydro Online Judge 进行提交和自动评测。

本地环境和 OJ 使用相同的 SPL Lexer、Framework 和评分逻辑。区别在于, 本地提供公开测试用于调试, 而 OJ 会使用额外的隐藏测试进行评分。

因此, 请先在本地充分测试, 再提交到 Hydro。

8.1 访问与登录

请访问以下链接进入课程 Hydro OJ: https://oj.sqlab.fun/d/cs323_2026fall/domain/join?code=cs323。

初始用户名和密码均为你的**学生学号**。登录后, 如果系统提示填写邀请码, 请输入 `cs323`。

请在**首次登录后及时修改初始密码**。点击页面右上角的用户名, 再进入**安全设置**即可修改密码。

8.2 如何提交

进入课程 Hydro OJ 后, 打开 **Project 1 - Lexer** 对应的题目。

提交注意事项

请按以下步骤填写提交页面:

1. 在“代码语言”下拉框中选择 **CS323 Project 1 Lexer**;
2. 将“代码”编辑框**保持为空, 不要填写任何内容**;
3. 点击“或者上传一个文件”旁的 **Choose File** 按钮, 选择并上传 `SplLexer.g4` 文件;
4. 确认代码语言和上传的文件正确后, 点击“递交”。

代码语言

CS323 Project 1 Lexer

1. 选择正确的代码语言

代码

2. 此处不要填写任何内容

或者上传一个文件: Choose File no file selected

3. 点击此处上传代码文件 (SplLexer.g4 文件)

提交

图 1: Project 1 的 OJ 提交页面及注意事项

注: 你只需要上传一个 `SplLexer.g4` 文件。

8.3 提交前检查

提交之前, 请确认:

1. 文件名为 `SplLexer.g4`;
2. `grammar` 声明仍然为:

```
lexer grammar SplLexer;
```

并确认:

- `make` 可以正常完成;
- `make test` 可以通过所有公开测试;
- 你已经根据本 Project 的规范自行测试了主要边界情况。

8.4 OJ 如何处理你的提交

上传 `SplLexer.g4` 后, 评测服务器会自动完成后续工作:



运行隐藏测试



计算得分

这些步骤与你在本地构建 Project 时执行的流程保持一致。因此, 你不需要关心 OJ 内部如何安装 ANTLR、生成 Java 文件或打包 Project。

8.5 本地测试与隐藏测试

本地运行:

```
make test
```

时, 如果某个公开测试失败, 会显示具体的输出差异, 方便你定位问题。这些公开测试和具体差异仅用于本地调试。

Hydro 中的隐藏测试则用于最终评分。

本 Project 满分为 100 分, 共包含 20 个隐藏测试样例, 每个测试样例 5 分。

Hydro 会根据通过的隐藏测试情况返回本次提交的得分。

为了保证隐藏测试的有效性, OJ 不会向学生公开隐藏测试对应的:

- 测试输入;
- 标准输出;
- 你的实际输出;
- 标准输出与实际输出之间的具体差异。

因此, 请使用本地公开测试和自己构造的测试完成调试, 而不要依赖反复提交 OJ 来推测隐藏测试内容。为了防止恶意高频提交, 每个账号一分钟内最多提交一次。

8.6 常见问题

本地 make 失败

首先查看终端中的 ANTLR 或 Java 编译错误。

常见原因包括:

- Lexer Rule 的 ANTLR 语法错误;
- 修改了 grammar 名称;
- Rule 名称拼写错误;
- `SplLexer.g4` 文件内容不完整。

请先解决本地构建问题, 再提交 OJ。

make test 中出现 FAIL

本地公开测试会显示具体的输出差异。

请重点检查:

- Token 类型名称是否正确;
- Keyword 和 Identifier 的 Rule 顺序;

- Longest Match;
- 多字符 Operator;
- Number;
- Whitespace 和 Comment 是否正确使用 skip。

公开测试全过，但 OJ 没有满分

公开测试无法覆盖所有边界情况。

请重新检查本 Project 的完整词法规范，并自行测试例如：

```
int integer int_value
= == < <= > >= ! !=
01234
123abc
123_awe
```

以及：

- 文件末尾没有换行的单行注释；
- 跨越多行的块注释；
- 多个 Operator 连续出现；
- Keyword 与 Identifier 的边界；
- 不同空白字符的组合。

是否需要提交生成的 SplLexer.java ?

不需要。

OJ 会根据你上传的 SplLexer.g4 重新运行 ANTLR。本地生成的 Java 文件不会参与提交或评分。

是否需要上传整个 Project ?

不需要。

本 Project 最终只提交 SplLexer.g4。课程提供的 Framework、ANTLR 依赖和隐藏测试均已经配置在评测环境中。

9 评分要求

评测将检查你的 Lexer 是否能够正确处理本项目定义的各类词法元素，包括：

- Keywords;
- Identifier;
- Number;
- Operators;
- Separators;
- Whitespaces;

- Line Comment;
- Block Comment;
- Longest Match;
- Rule Priority;
- 多种 Token 混合出现的 SPL 代码。

对于每一个测试样例，你需要保证：

1. 每一个 Token 的类型正确；
2. Token 类型名称与本 Project 规定的名称完全一致；
3. 每一个 Token 对应的 Lexeme 与源程序中的原始文本完全一致；
4. Token 的顺序与其在源程序中出现的顺序一致；
5. 空白符和注释被正确跳过。

10 示例

10.1 示例 1：

输入

```
int main() {
    int a[2 + 3];

    a[0] = 2 ^ 3;

    return a[0];
}
```

输出

```
Token: INT, Raw: int
Token: Identifier, Raw: main
Token: LPAREN, Raw: (
Token: RPAREN, Raw: )
Token: LBRACE, Raw: {
Token: INT, Raw: int
Token: Identifier, Raw: a
Token: LBRACK, Raw: [
Token: Number, Raw: 2
Token: PLUS, Raw: +
Token: Number, Raw: 3
Token: RBRACK, Raw: ]
Token: SEMI, Raw: ;
Token: Identifier, Raw: a
Token: LBRACK, Raw: [
Token: Number, Raw: 0
Token: RBRACK, Raw: ]
Token: ASSIGN, Raw: =
Token: Number, Raw: 2
Token: POW, Raw: ^
Token: Number, Raw: 3
Token: SEMI, Raw: ;
```

```
Token: RETURN, Raw: return
Token: Identifier, Raw: a
Token: LBRACK, Raw: [
Token: Number, Raw: 0
Token: RBRACK, Raw: ]
Token: SEMI, Raw: ;
Token: RBRACE, Raw: }
Token: EOF, Raw: <EOF>
```

10.2 示例 2:

注释和空白均被跳过，但注释两侧的文本不会拼接为一个 Token。块注释在最近的结束标记处结束，且不支持嵌套；因此本例中的 + 2 仍会被输出。

输入

```
int/* type */value = 8 / 2; // ignore the rest
/* first */value/* second */= 3;
ret/**/urn
/* outer
    /* inner */ + 2
```

输出

```
Token: INT, Raw: int
Token: Identifier, Raw: value
Token: ASSIGN, Raw: =
Token: Number, Raw: 8
Token: DIV, Raw: /
Token: Number, Raw: 2
Token: SEMI, Raw: ;
Token: Identifier, Raw: value
Token: ASSIGN, Raw: =
Token: Number, Raw: 3
Token: SEMI, Raw: ;
Token: Identifier, Raw: ret
Token: Identifier, Raw: urn
Token: PLUS, Raw: +
Token: Number, Raw: 2
Token: EOF, Raw: <EOF>
```